M03 - Advanced SQL + PostgreSQL / Database Thinking

16-lesson beginner-sufficient learner book - RC1

How to use each lesson

Read the explanation, reproduce the small example, complete the matching guided lab, then close the book for the independent task. A lesson is not mastered merely because the example works.

DE03-01 - INNER JOIN and LEFT JOIN

Why this matters	Joins combine related tables; join type changes which rows survive.

Core explanation

• INNER keeps matched rows.

• LEFT keeps every left-side row plus matches.

• Join on keys that represent the intended relationship.

• State input and output grain.

Concrete example

```
orders o LEFT JOIN customers c ON o.customer_id=c.customer_id
```

Common failure	Do not choose join type from habit; choose it from the question.
Proof before move-on	Return all orders with customer segment, including unmatched customers.

Explain aloud

In 60-90 seconds explain inner join and left join, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-02 - Cardinality, row multiplication and join validation

Why this matters	Many SQL errors come from hidden one-to-many relationships.

Core explanation

• Check key uniqueness before joining.

• One-to-many joins legitimately multiply left rows.

• Many-to-many may explode rows unexpectedly.

• Validate row counts and distinct keys after joins.

Concrete example

Count rows and distinct order_id before and after joining order_lines.

Common failure	Do not fix multiplication with DISTINCT until you understand the relationship.
Proof before move-on	Prove whether a join changed order grain.

Explain aloud

In 60-90 seconds explain cardinality, row multiplication and join validation, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-03 - UNION and UNION ALL

Why this matters	Set operations stack compatible result sets.

Core explanation

• UNION ALL keeps every row.

• UNION performs duplicate elimination.

• Columns must align by count and compatible meaning.

• Add source labels when combining event streams.

Concrete example

```
SELECT id,event_date,'Order' type FROM orders UNION ALL SELECT id,event_date,'Ticket' FROM tickets
```

Common failure	Do not use UNION when duplicate events are legitimate.
Proof before move-on	Build an activity stream and explain why UNION ALL is correct.

Explain aloud

In 60-90 seconds explain union and union all, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-04 - String functions for cleaning and standardization

Why this matters	SQL can standardize labels close to transformation logic.

Core explanation

• TRIM removes surrounding whitespace.

• UPPER/LOWER normalize case.

• CONCAT builds labels.

• LIKE handles patterns but can be expensive on large data.

Concrete example

UPPER(TRIM(region)) AS region_clean

Common failure	Do not overwrite source meaning without preserving original values.
Proof before move-on	Return cleaned names plus a readable label.

Explain aloud

In 60-90 seconds explain string functions for cleaning and standardization, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-05 - Dates, timestamps and time analysis

Why this matters	Time analysis requires explicit calendar and timestamp meaning.

Core explanation

• DATE and timestamp types are different.

• Date differences answer elapsed-time questions.

• Time zones can change calendar dates.

• Filter complete periods intentionally.

Concrete example

Compute days_to_return between order_date and return_date.

Common failure	Do not mix order date and refund date without declaring the business time basis.
Proof before move-on	Find the longest return delay and explain the time basis.

Explain aloud

In 60-90 seconds explain dates, timestamps and time analysis, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-06 - Subqueries and nested logic

Why this matters	Subqueries can express layered questions when the inner result is independently testable.

Core explanation

• Scalar subqueries return one value.

• Correlated subqueries run in relation to outer rows.

• Validate the inner result first.

• Prefer readability over clever nesting.

Concrete example

Filter completed orders above the completed-order average.

Common failure	Do not hide three business rules inside one unreadable expression.
Proof before move-on	Solve above-average logic and inspect inner/outer results separately.

Explain aloud

In 60-90 seconds explain subqueries and nested logic, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-07 - CTEs and readable multi-step SQL

Why this matters	CTEs name intermediate logic and make grain changes visible.

Core explanation

• WITH defines named query steps.

• Each CTE should have a clear grain.

• CTEs improve structure, not automatically speed.

• Validate each step while building.

Concrete example

```
WITH order_sales AS (...) SELECT * FROM order_sales WHERE net_sales>1000;
```

Common failure	Do not create ten CTEs if two clear steps are enough.
Proof before move-on	Rewrite a nested query as readable CTEs.

Explain aloud

In 60-90 seconds explain ctes and readable multi-step sql, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-08 - Window functions and PARTITION BY

Why this matters	Windows calculate across related rows without collapsing detail.

Core explanation

• GROUP BY reduces rows; windows usually preserve them.

• PARTITION BY creates calculation groups.

• ORDER BY inside OVER defines sequence.

• Window frames matter for running calculations.

Concrete example

AVG(net_sales) OVER(PARTITION BY region) AS region_avg

Common failure	Do not assume ORDER BY outside the query defines window order.
Proof before move-on	Add regional averages beside each order and flag above-average rows.

Explain aloud

In 60-90 seconds explain window functions and partition by, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-09 - ROW_NUMBER, ranking and LAG/LEAD

Why this matters	Analytic windows solve latest-row, ranking and previous/next comparisons.

Core explanation

• ROW_NUMBER needs deterministic ordering.

• RANK/DENSE_RANK handle ties differently.

• LAG/LEAD reference neighboring ordered rows.

• Partition by the entity whose sequence you want.

Concrete example

ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY created_at DESC, ticket_id DESC)

Common failure	Do not omit tie-breakers when "latest" must be deterministic.
Proof before move-on	Return latest ticket per customer and one previous-order interval.

Explain aloud

In 60-90 seconds explain row_number, ranking and lag/lead, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-10 - Advanced aggregation and multi-grain reasoning

Why this matters	Real queries often move through several grains before final output.

Core explanation

• Name the grain after every major step.

• Ratios need numerator and denominator at compatible grains.

• Distinct counts may be necessary for entity rates.

• Reconcile intermediates.

Concrete example

returned_orders / orders_count requires both counts at the same region-period grain.

Common failure	Do not divide event rows by order rows without confirming definitions.
Proof before move-on	Calculate a return rate and document numerator/denominator grain.

Explain aloud

In 60-90 seconds explain advanced aggregation and multi-grain reasoning, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-11 - PostgreSQL setup, schemas and dialect awareness

Why this matters	Data engineers often meet PostgreSQL; SQL concepts transfer but syntax and objects differ.

Core explanation

- PostgreSQL uses databases and schemas as namespaces.
- search_path affects unqualified object lookup.
- LIMIT is supported; identifier quoting uses double quotes.
- Use current docs for version-specific admin commands.

Concrete example

```
CREATE SCHEMA training; CREATE TABLE training.orders (...);
```

Common failure	Do not assume MySQL-only syntax works unchanged in PostgreSQL.
Proof before move-on	Translate one MySQL-flavored query/setup into PostgreSQL-safe syntax.

Explain aloud

In 60-90 seconds explain postgresql setup, schemas and dialect awareness, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-12 - Constraints and transactions

Why this matters	Database rules can prevent invalid states, and transactions group changes atomically.

Core explanation

• PRIMARY KEY enforces row identity.

• FOREIGN KEY protects references when enabled/enforced.

• CHECK and NOT NULL encode validity rules.

• BEGIN/COMMIT/ROLLBACK control transactional changes.

Concrete example

```
BEGIN; UPDATE inventory SET qty=qty-1 WHERE sku=...; COMMIT;
```

Common failure	Do not rely on application code alone for every invariant.
Proof before move-on	Design three table constraints and explain a transaction rollback case.

Explain aloud

In 60-90 seconds explain constraints and transactions, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-13 - Indexes and EXPLAIN basics

Why this matters	Indexes trade write/storage cost for faster access patterns; query plans provide evidence.

Core explanation

• Index keys should match actual filter/join patterns.

• More indexes are not automatically better.

• EXPLAIN shows plan choices; EXPLAIN ANALYZE executes in PostgreSQL.

• Measure before and after on realistic data.

Concrete example

```
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
```

Common failure	Do not claim an index helped without plan/runtime evidence.
Proof before move-on	Choose one index for a stated query and justify it.

Explain aloud

In 60-90 seconds explain indexes and explain basics, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-14 - Views and reusable database logic

Why this matters	Views package stable query logic behind a named interface.

Core explanation

- A view stores query definition, not ordinary copied rows.
- Views can centralize approved logic.
- Materialized views store results and require refresh.
- Permissions can expose curated columns while hiding raw detail.

Concrete example

```
CREATE VIEW reporting.completed_order_sales AS SELECT ...;
```

Common failure	Do not hide unclear grain inside a view and call it governance.
Proof before move-on	Design one reporting view with explicit grain and owner.

Explain aloud

In 60-90 seconds explain views and reusable database logic, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-15 - Integrated unseen SQL challenge

Why this matters	This lesson tests cross-topic transfer without a worked template.

Core explanation

• Inspect schema first.

• Plan joins and grain on paper.

• Build in validated steps.

• Save checks beside the final query.

Concrete example

Scenario: identify the top two customers by completed sales within each region while retaining deterministic tie handling.

Common failure	Do not open review material during the first attempt.
Proof before move-on	Save final SQL plus row-count, key and total reconciliation evidence.

Explain aloud

In 60-90 seconds explain integrated unseen sql challenge, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE03-16 - Timed SQL interview set and explanation

Why this matters	Interview readiness requires correct SQL plus clear verbal reasoning.

Core explanation

• Time-box the first attempt.

• Prefer simple correct queries over clever ones.

• Explain grain, join/cardinality risk and validation.

• Retest failed skills with a fresh question.

Concrete example

Complete four mixed questions in 35-45 minutes, then explain one solution aloud in under 2 minutes.

Common failure	Do not memorize the review query and call it mastery.
Proof before move-on	Pass the timed set or complete targeted repair plus fresh retest.

Explain aloud

In 60-90 seconds explain timed sql interview set and explanation, the input/output grain or program state, and one way a plausible-looking result could still be wrong.